

LO2.4 and LO3: Test Results

```
[INFO] Results:
[INFO]
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 17.106 s
[INFO] Finished at: 2026-01-20T15:29:49Z
[INFO] -----

Process finished with exit code 0
```

LO2.4: The test plan has been successfully actioned, with all planned tests executed and passing for the implemented requirements, providing evidence the system behaves as intended for the selected high priority requirements.

For R1, the tests check that the system does not violate the safety requirement under realistic and edge case conditions. These include cases where a direct path is blocked and in the case that the drone is blocked in all directions by a restricted area. This thorough verification that the system will never violate the safety protocol under any conditions. The inclusion of synthetic area data containing unusual situations (like the drone being contained entirely within the restricted area) reduces the risk mentioned in the test plan, and allows confidence in the safety features of the system early on.

For R2, tests were carried out at the end of the development process, as decided in the test plan. These tests use small and larger amounts of synthetic input to ensure the system performs well under different conditions. This provides evidence the system meets the time requirement in a controlled environment, follows the test plan, and provides some mitigation for the risk mentioned in the test plan. Doing this testing after everything else has been implemented ensures the software meets the measurable requirements and is ready for use.

For R3, unit level tests were used to verify the drone movement constraints defined in the specification. Validation tests confirm that only valid compass directions (multiples of 22.5 degrees) are accepted, and movement tests verify that each step the drone takes is the fixed step size 0.00015 degrees in the right direction. These tests demonstrate that the fundamental movement rules are enforced consistently and deterministically. The same movement logic is exercised indirectly during path generation tests, providing additional confidence that these constraints are respected during normal system operation.

Overall, the execution of the test plan shows that the testing effort was applied in line with requirement priority and lifecycle placement. Safety critical and deterministic requirements were verified early, while performance constraints were validated once the system was fully integrated, allowing for an organised, risk aware testing process.

LO3.1: A large range of different techniques have been applied here across unit, integration and system levels, with good consistency with the test plan with some tests applied beyond the test plan when risks became apparent during development.

Functional testing (black-box testing) was used extensively for unit level testing, including tests for the /distanceTo, /isCloseTo, /nextPosition and /isInRegion endpoints. These tests check the outcomes of these endpoints, including the HTTP status codes and responses to ensure that they match valid outcomes, without thoroughly examining the inner workings of each function. These tests exercise equivalence classes and boundary conditions derived from the specification.

Informal category partition-style testing was used in the geometry and validation tests. Inputs were partitioned into valid/invalid positions, valid/invalid angles, etc. Test cases were created to represent each of these categories to ensure the tests cover important behaviours while keeping the amount of tests at a reasonable level.

Integration testing was used for request handling and path related behaviour by combining controllers, validation and geometry services using @SpringBootTest with mocked external dependencies. This ensured that independently tested components interacted correctly.

System level and performance testing was applied to calcDeliveryPath using an instrumented test with mocked endpoints for different requests. Execution time constraints were checked using assertTimeoutPreemptively, providing evidence for the measurable performance requirement.

L.O3.2: Different criteria for adequacy were applied depending on the testing technique used and the requirements being tested.

For the functional and unit tests, adequacy was evaluated using equivalence class coverage and boundary value coverage. Each identified class of valid and invalid input is exercised at least once, giving confidence in especially the safety critical operations like region checking and movement calculations.

For structural tests, adequacy was informally evaluated through branch coverage, checking that both acceptance and rejection paths in validation and geometry logic were executed by the tests. This reduces the risk that faults are hidden in unexecuted branches of critical code.

For integration testing, adequacy was evaluated by the ability of the tests to detect faults in interaction, such as mismatches between path generation and restricted area data or invalid request propagation across layers.

For performance testing, adequacy was evaluated against the explicit performance requirement "Route computation must complete within 30 seconds". While this criteria is strict and unexplained, it is appropriate given the specification and the use of mocked, repeatable workloads.

These criteria are optimistic, they assume representative tests data and do not guarantee the absence of faults. However, they provide a pragmatic level of confidence aligned with the project scope.

LO3.3: The functional and integration tests implemented passed consistently, demonstrating correct handling of valid requests and rejection of invalid inputs. Several issues were identified during testing,

particularly around request validation and edge cases in geometry calculations, such as the drone getting stuck with restricted objects due to how angles were picked, which were corrected iteratively.

Performance testing using mocked endpoints showed that `calcDeliveryPath` completes well within the 30 second limit for both small and larger synthetic workloads of up to 50 medical dispatch records. This shows that the implementation satisfies the specified performance constraint under controlled conditions.

L.O3.4: Overall, this testing has increased confidence that the implemented system meets its functional, safety and performance requirements. The combination of functional, structural, integration, and performance testing reduces the likelihood of systematic faults in critical components such as movement logic and restricted-area avoidance.

However, limitations remain. Test data for performance and integration testing is synthetic and may not reflect all real-world scenarios. Structural coverage was assessed informally rather than through automated coverage tools, and some complex interactions, like large numbers of drones or no fly zones, were not tested to the extreme.

Addressing these limitations would require more extensive automated coverage analysis, larger-scale performance testing, and potentially model-based testing of pathfinding behaviour. Within the constraints of the project, the testing performed fits well, and offers a large amount of confidence in how the system will perform in real-world conditions.